Módulo 5 – Testes de Performance com K6

Nome: Mariana Caetano Costa
Email: mariana.cae.costa@gmail.com
Telefone: (31) 9.8859-0581

OBJETIVO DO MÓDULO

Aprender sobre automação de testes de performance e implementar testes com k6 e Javascript.

---

1. PREPARAÇÃO DO AMBIENTE

Durante o módulo, o ambiente utilizado para os testes foi composto pelos seguintes recursos:

Banco API

• Git Bash: acesso à pasta **banco-api**
• Inicialização do projeto pelo comando: **npm run rest-api**
• Acesso à aplicação pelo navegador.

Banco Web

• Git Bash: acesso à pasta **banco-web**
• Inicialização do servidor pelo comando: **npm run server**
• Acesso ao sistema pelo navegador.

Banco de Dados MySQL

• Banco de dados utilizado: banco.
• Usuário: julio.lima (acesso total)
• Usuário: junior.lima (acesso limitado)

---

2. MENTALIDADE DE TESTES DE PERFORMANCE

O objetivo desta aula é entender como um QA deve pensar, planejar e avaliar testes de performance, considerando não apenas a parte técnica, mas também o comportamento dos usuários, os riscos do sistema e o impacto dos resultados para o negócio.

1. O que são testes de performance?

Testes de performance avaliam o comportamento de um componente ou de um sistema completo quando submetido a determinada carga de trabalho.

Essa carga pode ser:

- **Esperada**: representa o uso normal do sistema no dia a dia.
- **Acima da esperada**: simula uma quantidade maior de usuários ou requisições.
- **Elevada por um período prolongado**: verifica como o sistema se comporta sob carga contínua.
- **Muito alta em um intervalo curto:** verifica a reação do sistema diante de picos de demanda.

O objetivo é identificar limitações, gargalos e problemas de desempenho antes que afetem os usuários reais.

2. Onde os testes de performance são aplicados?
São muito utilizados em aplicações web, APIs, serviços e bancos de dados.
Em uma arquitetura cliente-servidor, por exemplo, o cliente envia solicitações, o servidor processa as informações, consulta ou grava dados no banco e devolve uma resposta ao cliente.



O teste pode ajudar a descobrir se a lentidão está relacionada ao servidor, à API, ao banco de dados, à infraestrutura ou à interação entre esses componentes.

3. Exemplo prático: lançamento de um game

Imagine uma loja virtual que venderá um jogo muito aguardado.
Durante o lançamento, milhares de pessoas podem tentar acessar o site, comprar o jogo ou iniciar o download simultaneamente.
O QA precisa investigar se o sistema suporta essa demanda.

Cenário de teste	**Resultado hipotético**
500 usuários simultâneos durante 3 horas	Aplicação funciona dentro dos critérios definidos
600 usuários simultâneos	Tempo de resposta aumenta significativamente
Pico de acessos acima da capacidade	Podem ocorrer erros, indisponibilidade ou falhas nas requisições

Os números são exemplos da aula, não limites universais de desempenho.

Se o sistema apresentar problemas, a equipe de engenharia poderá investigar melhorias de memória, processamento, armazenamento, configurações, software ou arquitetura.

Papel do QA: identificar e documentar o problema, apresentar evidências e ajudar a equipe a compreender em quais condições o sistema deixa de atender aos requisitos.

4. O que deve passar pela mente de um QA?
O instrutor destaca diferentes aspectos que precisam ser considerados no planejamento.

4.1. Riscos de desempenho

Pergunte-se:

- O sistema suporta a quantidade esperada de usuários?
- Existe risco de indisponibilidade?
- O tempo de resposta aumenta com o crescimento da carga?
- Em que ponto o sistema começa a apresentar falhas?

4.2. Comportamento dos usuários

Não basta simular muitos usuários acessando uma página. É necessário reproduzir jornadas que representem o uso real do sistema.

Em uma loja virtual, por exemplo, os usuários podem:

- Acessar a página do produto.
- Consultar detalhes e disponibilidade.
- Adicionar produtos ao carrinho.
- Escolher uma forma de pagamento.
- Finalizar a compra e consultar o recibo.
- Fazer o download de um produto digital.

Os cenários precisam representar a combinação de ações esperada no sistema. Se todos os usuários executarem exatamente a mesma ação, o teste poderá não representar adequadamente o comportamento real.

4.3. Dados de teste

É necessário planejar quais dados serão utilizados pelos usuários virtuais.
Exemplo: para simular 50 usuários realizando operações que exigem contas diferentes, pode ser necessário preparar 50 conjuntos de dados ou definir uma estratégia de reutilização compatível com as regras da aplicação.

Considere:

- Usuários e credenciais.

- Produtos e identificadores.
- Dados de compra.
- Dados exclusivos ou compartilhados.
- Limpeza e preparação dos dados antes e depois da execução.

A quantidade exata de registros depende do cenário e das regras do sistema.

4.4. Escolha da ferramenta
O QA deve escolher uma ferramenta compatível com a aplicação, os protocolos utilizados, os cenários planejados e os resultados que precisa analisar.

Entre as ferramentas comuns nesse universo estão:
- **Apache JMeter**: testes de carga em aplicações e APIs.
- **k6:** testes de performance com scripts e métricas.
- **Gatling:** testes de carga automatizados.

A escolha não deve ocorrer apenas porque uma ferramenta é popular. É preciso entender por que ela atende à necessidade do projeto.

4.5. Critérios para avaliar o sucesso
Antes de executar os testes, o QA precisa saber quais condições caracterizam um resultado satisfatório. Essas condições são definidas nos requisitos não funcionais de performance e nos critérios de aceitação acordados com a equipe.

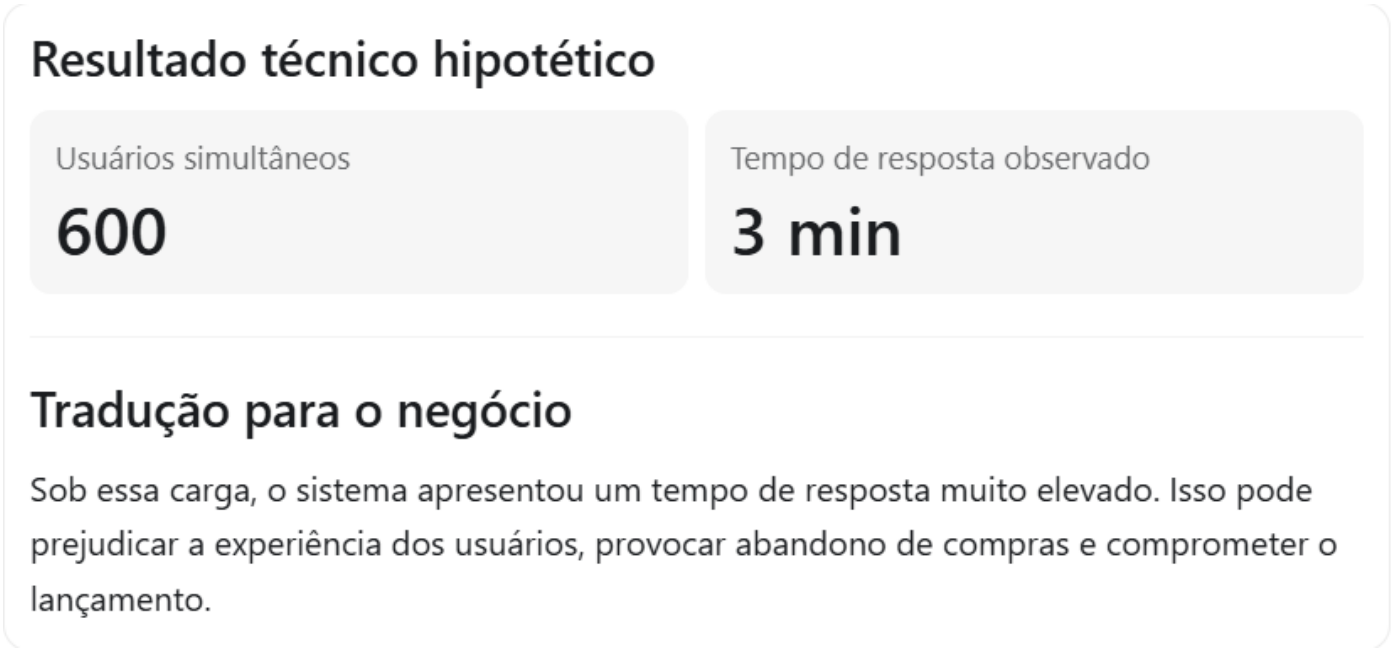
Exemplos hipotéticos:
- 95% das requisições devem responder em até 2 segundos.
- A taxa de erros deve permanecer abaixo de 1%.
- O sistema deve sustentar 500 usuários simultâneos durante 3 horas.
- As operações críticas devem continuar funcionando durante a carga prevista.
-

Esses valores são apenas exemplos. Os limites reais devem ser definidos conforme a necessidade do negócio.

5. Como interpretar e comunicar os resultados
Uma parte fundamental da mentalidade de performance é transformar resultados técnicos em informações úteis para pessoas que tomam decisões.

Os relatórios podem conter muitas métricas, tabelas e gráficos. Não basta apresentar esses dados sem explicar seu significado.

Considere este exemplo:



Essa tradução ajuda gestores, desenvolvedores e responsáveis pelo produto a compreenderem o impacto do problema e a priorizarem ações corretivas.

6. Checklist de planejamento de performance

Antes de executar um teste, use esta lista como referência:

Checklist do QA
0 de 8

- Identificar os riscos de desempenho
- Definir a carga esperada e os cenários de pico
- Mapear as jornadas dos usuários
- Preparar os dados de teste
- Escolher a ferramenta e justificar a escolha
- Definir os requisitos e critérios de sucesso
- Monitorar métricas e comportamento do sistema
- Documentar os resultados e comunicar o impacto no negócio

7. Principais aprendizados da aula

- Testes de performance verificam o comportamento do sistema sob diferentes cargas.
- O planejamento deve considerar riscos, comportamento dos usuários, dados, ferramentas e critérios de sucesso.
- A execução precisa representar cenários relevantes para o uso real.
- Os resultados devem ser analisados tecnicamente e traduzidos em impactos para o negócio.
- O QA não precisa apenas identificar lentidão: deve produzir evidências que ajudem a equipe a entender o problema e decidir como agir.

Conclusão:

Nesta aula, o foco não é aprender uma ferramenta específica, mas desenvolver a mentalidade necessária para planejar testes de performance que representem o uso real e produzam resultados úteis para a tomada de decisão.

---

3. PERFORMANCE E A ISO-25010

1. O que é a ISO 25010?
A ISO/IEC 25010 é uma norma internacional que define um modelo de qualidade de software, descrevendo características utilizadas para avaliar a qualidade de um produto de software.

Na aula, o instrutor apresenta a eficiência de desempenho (Performance Efficiency) como uma das características de qualidade analisadas nos testes não funcionais.

2. Diferença entre testes funcionais e testes de performance
A principal diferença está no objetivo da avaliação.

Cenário de login	**Tipo de teste**	**O que avaliar**
Login com credenciais válidas	Funcional	O usuário consegue entrar?
Login com senha incorreta	Funcional	O sistema rejeita as credenciais?
Login com senha expirada	Funcional	O sistema aplica a regra prevista?
Tempo para concluir o login	Performance	Quanto tempo a operação demora?
Consumo de memória durante o login	Performance	Quanta memória a aplicação utiliza?
Quantidade de usuários suportada	Performance	Qual é a capacidade do sistema?

Conceito importante: no teste funcional, verificamos se a funcionalidade funciona corretamente. No teste de performance, avaliamos sua eficiência sob determinadas condições.

Uma funcionalidade pode estar correta e, mesmo assim, apresentar um tempo de resposta inaceitável.

3. Eficiência de desempenho segundo a ISO 25010

A eficiência de desempenho é apresentada na aula por meio de três subcaracterísticas:

1. Comportamento temporal (Time Behaviour)
Avalia se os tempos de resposta, os tempos de processamento e a vazão (throughput) atendem aos requisitos estabelecidos.

Exemplo: verificar se 95% das requisições de login respondem em até 2 segundos, conforme o requisito definido.

2. Utilização de recursos (Resource Utilization)

Avalia a quantidade e os tipos de recursos utilizados pelo sistema durante sua execução, como CPU, memória e conexões disponíveis.

Exemplo: verificar se o consumo de CPU aumenta excessivamente quando cresce o número de usuários simultâneos.

3. Capacidade (Capacity)
Avalia se os limites máximos de um sistema ou componente atendem aos requisitos estabelecidos.

Exemplo: verificar se uma aplicação suporta 500 usuários simultâneos mantendo os tempos de resposta exigidos.

Observação: os exemplos numéricos são ilustrativos. Os limites reais devem vir dos requisitos do sistema.

4. Aplicação prática no trabalho de QA

Ao planejar um teste de performance, o QA pode investigar:

- Tempo de resposta: quanto tempo uma operação leva para terminar.
- Vazão (throughput): quantas requisições o sistema processa por unidade de tempo.
- Consumo de recursos: quanto de CPU, memória e outros recursos é utilizado.
- Capacidade: qual volume de usuários ou requisições o sistema suporta dentro dos critérios definidos.

Essas métricas ajudam a identificar gargalos e a verificar se a aplicação atende às expectativas de desempenho.

5. O que guardar desta aula

- A ISO 25010 apresenta características de qualidade de software.
- A eficiência de desempenho é uma característica avaliada em testes não funcionais.
- Suas três subcaracterísticas são comportamento temporal, utilização de recursos e capacidade.
- Testes funcionais verificam o comportamento das funcionalidades; testes de performance avaliam sua eficiência.
- Uma aplicação pode funcionar corretamente e ainda apresentar problemas de desempenho.

Conclusão:
A ISO 25010 apresenta a eficiência de desempenho como uma característica da qualidade de software, avaliada por meio de três subcaracterísticas: comportamento temporal, utilização de recursos e capacidade. O QA deve verificar se o sistema atende aos requisitos de desempenho estabelecidos, identificando possíveis gargalos e avaliando seu comportamento sob diferentes condições de uso.

---

4. REQUISITOS NÃO FUNCIONAIS DE PERFORMANCE
1. O que são requisitos não funcionais de performance?

Os requisitos não funcionais de performance definem como o sistema deve se comportar em relação à eficiência de desempenho, estabelecendo critérios como tempo de resposta, capacidade de processamento e métricas que precisam ser atendidas.

Enquanto os testes funcionais verificam se uma funcionalidade funciona corretamente, os testes de performance avaliam se ela apresenta o desempenho esperado.

2. A importância de documentar os requisitos

O instrutor relata que, mesmo em empresas consolidadas, os requisitos de performance podem não estar documentados formalmente. Em seu exemplo, os critérios eram transmitidos verbalmente pela equipe:

- As requisições à API deveriam responder em até 2 segundos (2.000 milissegundos).
- O critério deveria ser avaliado pelo percentil 90 (P90).

Embora essas informações permitam iniciar os testes, elas não são suficientes para definir todos os critérios de aceitação.

É necessário esclarecer, por exemplo:

- Qual percentil deve ser considerado: P90, P95 ou P99?
- O limite de tempo se aplica a todos os endpoints?
- Algumas operações precisam responder mais rapidamente?
- Existem operações que podem demorar mais devido à sua natureza?
- Qual é a prioridade desse requisito para o negócio?

Conceito-chave: um requisito de performance precisa ser claro, mensurável e adequado à funcionalidade avaliada.

3. Exemplo de requisito não funcional (NFR)

O instrutor apresenta um exemplo de documentação de requisitos não funcionais, utilizando o identificador NFR (Non-Functional Requirement).

Campo	**Exemplo**
ID	NFR-001
Descrição	O sistema deve responder em até 2 segundos para pelo menos 95% das requisições, sob as condições de carga definidas.
Prioridade	Alta
Métrica	Tempo de resposta
Forma de avaliação	Medição por ferramenta de teste de performance

O exemplo acima é ilustrativo. Na prática, o requisito também deve especificar as condições de carga, o ambiente de execução e a operação avaliada, para que o resultado possa ser reproduzido e interpretado corretamente.

4. Onde documentar os requisitos?

Os requisitos não funcionais podem ser registrados em diferentes ferramentas, conforme o processo de trabalho da empresa.

- Jira: na própria User Story, em uma seção específica para requisitos não funcionais e critérios de aceitação.
- Confluence: em uma base de conhecimento com requisitos organizados e documentados.
- Ferramentas de gestão de requisitos: em sistemas específicos utilizados pela organização.

O importante é que a equipe consiga consultar os requisitos antes de planejar e executar os testes.

5. Qual é o papel do QA?

O QA não precisa ser o único responsável por escrever os requisitos não funcionais. Entretanto, deve participar da conversa com o time de engenharia para esclarecer critérios ausentes ou ambíguos.

Entre suas responsabilidades estão:

- Identificar requisitos de performance inexistentes ou incompletos.
- Conversar com desenvolvedores, analistas e responsáveis pelo produto.
- Ajudar a transformar expectativas genéricas em critérios mensuráveis.
- Definir como os resultados serão avaliados em relação aos requisitos.
- Priorizar os testes conforme a importância e os riscos para o negócio.

A prioridade ajuda a determinar por onde começar quando existem muitos requisitos a avaliar e recursos limitados.

Conclusão

Os testes de performance precisam partir de requisitos não funcionais claros, documentados e mensuráveis. Não basta afirmar que o sistema deve ser rápido: é necessário estabelecer limites de tempo de resposta, percentis, condições de carga, operações avaliadas e prioridades.

O QA tem um papel importante nesse processo, identificando lacunas e promovendo conversas com o time para que os critérios sejam definidos e registrados. Com os requisitos formalizados, torna-se possível executar os testes com ferramentas adequadas e avaliar objetivamente se o sistema atende às expectativas de desempenho.

---

5. MÉTRICAS ESTATÍSTICAS

1. Por que as métricas estatísticas são importantes?

Durante os testes de performance, coletamos os tempos de resposta das requisições executadas pela aplicação. Como diferentes usuários podem obter tempos diferentes, precisamos utilizar métricas estatísticas para interpretar os resultados e comunicar o desempenho do sistema.

As principais métricas apresentadas nesta aula são:

- Mínimo;
- Máximo;
- Média;
- Desvio padrão;
- Mediana (P50);
- Percentil 90 (P90).

2. Exemplo utilizado na aula

O instrutor simula dez usuários realizando requisições ao endpoint POST /login, registrando os seguintes tempos de resposta:

Usuário	Tempo de resposta (ms)
1	100
2	200
3	200
4	100
5	300
6	1.000
7	800
8	700
9	700
10	200

Para facilitar a análise estatística, os valores são ordenados em ordem crescente:
100, 100, 200, 200, 200, 300, 700, 700, 800, 1000

A população analisada, neste exemplo, é o conjunto dos dez tempos de resposta registrados.

3. Principais métricas estatísticas

3.1. Mínimo (Minimum)

Representa o menor tempo de resposta observado durante a execução dos testes.

- Resultado: 100 ms.
- Interpretação: a requisição mais rápida foi concluída em 100 milissegundos.

3.2. Máximo (Maximum)

Representa o maior tempo de resposta observado.

- Resultado: 1.000 ms (1 segundo).
- Interpretação: a requisição mais lenta levou 1 segundo para ser concluída.

O mínimo e o máximo ajudam a compreender a amplitude dos resultados e a identificar valores extremos que merecem investigação.

3.3. Média (Average)

É calculada somando todos os valores e dividindo o resultado pela quantidade de observações.

$$\text{Média} = \frac{\text{Soma dos valores}}{\text{Quantidade}}$$

Neste exemplo:

$$\text{Média} = \frac{4300}{10} = 430 \text{ ms}$$

Resultado: 430 ms.

A média fornece uma visão geral dos tempos de resposta, mas pode ser influenciada por valores muito altos ou muito baixos.

Por exemplo, embora a média seja 430 ms, nenhum dos dez tempos registrados é exatamente igual a esse valor.

Por isso, a média não deve ser analisada isoladamente. É importante combiná-la com outras métricas para compreender melhor a distribuição dos tempos de resposta.

3.4. Desvio padrão (Standard Deviation)

O desvio padrão mede o quanto os valores estão dispersos em relação à média.

- Desvio padrão baixo: os valores tendem a estar mais próximos da média.
- Desvio padrão alto: os valores tendem a apresentar maior dispersão.

No exemplo da aula, considerando o desvio padrão populacional, o resultado é aproximadamente 282,49 ms.

Isso indica uma dispersão considerável dos tempos de resposta.

A média e o desvio padrão podem ser apresentados juntos para contextualizar os resultados. Entretanto, o intervalo entre média menos um desvio padrão e média mais um desvio padrão não garante que a maioria dos valores esteja nesse intervalo. Essa interpretação depende da distribuição dos dados.

4. Mediana — Percentil 50 (P50)

A mediana representa o valor central de um conjunto de dados ordenados. Ela divide as observações em duas metades.

No exemplo, temos dez valores. Como a quantidade é par, a mediana é a média entre o quinto e o sexto valores:

$$P50 = \frac{200 + 300}{2} = 250 \text{ ms}$$

Resultado: 250 ms.

Interpretação: aproximadamente 50% das observações ficaram em até 250 ms, e aproximadamente 50% ficaram em 250 ms ou mais.

A mediana é menos sensível a valores extremos do que a média. Por isso, pode oferecer uma visão mais representativa do centro da distribuição quando existem tempos de resposta muito elevados.

5. Percentil 90 (P90)

O percentil 90 indica um valor abaixo ou igual ao qual se encontra aproximadamente 90% das observações, conforme o método de cálculo utilizado.

No exemplo, utilizando o método do Excel PERCENTILE.INC, o resultado é:
P90 = 820 ms.

Interpretação: aproximadamente 90% das observações ficaram em até 820 ms, enquanto aproximadamente 10% ficaram acima desse valor.

Essa métrica é importante porque a média ou a mediana, sozinhas, podem esconder a experiência dos usuários que recebem respostas mais lentas.

O P90 também ajuda a avaliar requisitos de performance. Por exemplo, se um requisito estabelece que 90% das requisições devem responder em até 2 segundos, podemos comparar o P90 medido com esse limite, desde que as condições do teste correspondam às condições especificadas no requisito.

6. Comparação das métricas

Métrica	**Resultado do exemplo**	**O que representa**
Mínimo	100 ms	Menor tempo observado
Máximo	1.000 ms	Maior tempo observado
Média	430 ms	Soma dos tempos dividida por dez
Desvio padrão populacional	282,49 ms	Dispersão dos tempos em relação à média
P50 (mediana)	250 ms	Valor central da distribuição
P90	820 ms	Valor até o qual ficam aproximadamente 90% das observações

7. Uso das métricas nas ferramentas de performance

O Excel pode ser utilizado para compreender e calcular essas métricas durante os estudos. Porém, ferramentas de testes de performance, como JMeter, k6 e Gatling, fornecem relatórios com diferentes métricas estatísticas.

O QA precisa saber interpretar os resultados, identificar possíveis problemas e comunicar os achados à equipe. Não basta apenas apresentar números: é necessário relacioná-los aos requisitos e às condições em que o teste foi executado.

Conclusão
As métricas estatísticas permitem interpretar os tempos de resposta obtidos durante os testes de performance. O mínimo e o máximo mostram os extremos observados; a média resume os valores; o desvio padrão indica sua dispersão; a mediana representa o centro da distribuição; e os percentis ajudam a compreender o comportamento de uma parcela específica das requisições.
O principal aprendizado é não analisar a média isoladamente. Para avaliar a experiência dos usuários, especialmente aqueles que enfrentam respostas mais lentas, os percentis como P50, P90, P95 e P99 são muito úteis. O QA deve compreender o significado dessas métricas para comparar os resultados com os requisitos de desempenho e identificar oportunidades de melhoria.

---

6. INTRODUÇÃO E INSTALAÇÃO DO K6

1. O que é o k6?
O k6 é uma ferramenta de automação de testes de performance que permite escrever scripts de teste utilizando JavaScript.

Embora os scripts sejam escritos em JavaScript, a ferramenta k6 foi desenvolvida em Go e é mantida pela Grafana.

Ela permite simular requisições a aplicações e APIs para avaliar seu comportamento sob diferentes condições de carga.

2. Principais características do k6

- **JavaScript:** permite escrever os scripts de teste utilizando uma linguagem comum no desenvolvimento e na automação de testes.
- **Testes de API:** possibilita executar requisições HTTP, como GET e POST.
- **Verificações:** permite validar condições das respostas recebidas, como o código de status HTTP.
- **Pausas entre requisições:** oferece funções para controlar intervalos durante a execução dos scripts.
- **Execução local e na nuvem:** permite executar testes localmente e utilizar recursos de execução na nuvem, conforme a solução adotada.
- **Relatórios:** fornece métricas para analisar os resultados dos testes de performance.

3. Como funciona a automação com k6?

O fluxo básico de trabalho é:

1. Escrever um script de teste em JavaScript.
2. Utilizar as funções disponibilizadas pelo k6 para executar requisições.
3. Definir verificações para avaliar as respostas.
4. Executar o script pelo terminal.
5. Analisar as métricas e os resultados apresentados.

Diferentemente de bibliotecas JavaScript instaladas por meio do npm install, o k6 é instalado como uma ferramenta independente, normalmente por meio de um pacote ou instalador do sistema operacional.

Importante: o k6 não é uma ferramenta com interface gráfica tradicional. A criação dos scripts e sua execução são realizadas por meio de arquivos de código e do terminal.

4. Conhecendo as funções do k6

O instrutor apresenta um exemplo de script com algumas funções importantes:

Função	**Finalidade**
http.post()	Envia uma requisição HTTP POST.
check()	Verifica se uma ou mais condições são atendidas pela resposta.
sleep()	Introduz uma pausa na execução do script.
default function	Define a função principal executada pelo k6 em cada iteração de um usuário virtual.

Exemplo simplificado:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export default function () {
```

```
  const response = http.post(
    'https://exemplo.com/login',
    JSON.stringify({
      username: 'usuario',
      password: 'senha'
    }),
    {
      headers: {
        'Content-Type': 'application/json'
      }
    }
  );

  check(response, {
    'status é 200': (r) => r.status === 200
  });

  sleep(0.3);
}
```

Esse exemplo é ilustrativo: o endereço, os campos enviados e o código de status esperado precisam corresponder à API real que será testada.

A função check() verifica uma condição, mas, por si só, não interrompe a execução do teste quando ela falha.

5. Instalação do k6

A instalação depende do sistema operacional.

https://k6.io/
Open Source > downloand > OSS > Grafana K6 > Downloand

Windows

1. Crie a pasta k6
2. Abra o **gitbash** nela e faça a instalação: **winget install k6 --source winget**
3. Instalador o instalador oficial mais recente .

6. Como verificar se o k6 foi instalado?

Após a instalação, abra o terminal **GitBash** ou **Power Shell** e execute: **k6 version**

Se a instalação estiver correta e o comando estiver disponível no PATH, o terminal apresentará a versão instalada.

O comando também ajuda a confirmar se o ambiente está preparado para executar os primeiros scripts.

7. O que é necessário guardar desta aula?

- O k6 é uma ferramenta de testes de performance que utiliza JavaScript para escrever scripts.
- A ferramenta foi desenvolvida em Go e é mantida pela Grafana.
- Sua instalação é independente das dependências de um projeto npm.
- Os scripts podem realizar requisições HTTP, verificar respostas e controlar pausas.
- A execução é feita pelo terminal, e não por uma interface gráfica tradicional.
- Antes de começar os testes, é necessário instalar a ferramenta e confirmar sua disponibilidade com k6 version.

Conclusão

Nesta aula, você conheceu o k6, suas principais características e a forma como ele será utilizado na automação de testes de performance. Também aprendeu que os scripts são escritos em JavaScript, mas a ferramenta é instalada separadamente do projeto e executada pelo terminal.
Com o k6 instalado e funcionando, o próximo passo será criar os primeiros scripts, realizar requisições à aplicação e compreender como as verificações e as métricas ajudam a avaliar o desempenho do sistema.

---

7. ESCREVENDO O PRIMEIRO SCRIPT

1. Objetivo da aula

Criar o primeiro projeto de testes de performance utilizando o Grafana k6, organizando o código em um repositório GitHub, configurando o ambiente no Visual Studio Code e escrevendo um script JavaScript para testar o endpoint de login de uma API.

2. Criando o repositório no GitHub

O primeiro passo é criar um repositório para armazenar e versionar o projeto.
Exemplo utilizado na aula: banco-api-performance.

Etapas:

1. Criar um repositório público no GitHub.
2. Copiar a URL do repositório.
3. Abrir o Git Bash e acessar a pasta onde os projetos ficam armazenados.
4. Clonar o repositório.
5. Entrar na pasta do projeto.

Comandos utilizados:
git clone URL_DO_REPOSITORIO
cd banco-api-performance
ls

- git clone: copia o repositório remoto para o computador.

- cd: acessa um diretório.
- ls: lista os arquivos e as pastas do diretório atual.

O caminho utilizado para armazenar os projetos pode variar conforme a organização de cada pessoa.

3. Abrindo o projeto no Visual Studio Code

Após clonar o repositório, abrir a pasta do projeto no Visual Studio Code.
No terminal integrado, verificar a branch atual: **git branch**

Criar uma nova branch para iniciar o desenvolvimento: **git checkout -b iniciando-projeto**

Verificar novamente: **git branch**

A branch iniciando-projeto será utilizada para desenvolver o primeiro script, mantendo o trabalho organizado e separado da branch principal, main.

Observação: criar uma branch não envia automaticamente as alterações ao GitHub. Para publicar as alterações posteriormente, será necessário realizar o commit e o push.

4. Como funciona o k6 com JavaScript

O k6 permite escrever scripts de testes de performance em JavaScript, mas não funciona como um projeto JavaScript convencional executado pelo Node.js.

Características importantes:
- O k6 é uma ferramenta desenvolvida em Go.
- Os scripts de teste são escritos em JavaScript.
- A execução dos scripts é feita pelo comando k6.
- Não é necessário executar npm install apenas para utilizar os módulos nativos do k6.
- A ferramenta oferece módulos próprios para requisições HTTP, verificações e controle de tempo.

Verificar a versão instalada: **k6 version**

5. Criando a estrutura inicial do projeto

No Visual Studio Code, criar a seguinte estrutura:

```
banco-api-performance/
└── testes/
    └── login.test.js
```

A pasta testes armazenará os scripts de performance. O arquivo login.test.js será responsável pelo teste inicial do endpoint de login.

A estrutura poderá ser aprimorada conforme o projeto crescer.

6. Consultando a documentação oficial

A documentação deve ser uma das primeiras fontes consultadas ao aprender uma ferramenta nova.

Documentação oficial: grafana.com

Ela contém exemplos, referências de funções, configurações de execução e orientações para criar scripts.

A aula utiliza principalmente a seção de primeiros passos e a documentação de requisições HTTP.

7. Estrutura básica de um script k6

O script utiliza três elementos principais: importações, configurações e função de execução.

7.1. Importando os módulos

```
import http from 'k6/http';
import { sleep } from 'k6';
```

- http: módulo utilizado para realizar requisições HTTP, como GET e POST.
- sleep: função utilizada para aguardar um intervalo de tempo durante a execução do teste.

7.2. Configurando as opções do teste

A aula apresenta a constante options, exportada para que o k6 reconheça as configurações de execução.

É importante corrigir um detalhe técnico da explicação: a propriedade padrão utilizada pelo k6 para definir o número de iterações é iterations, e não interactions.

Exemplo de configuração:

```
export const options = {
  iterations: 10,
};
```

Nesse exemplo, o k6 executará dez iterações no total, considerando o cenário padrão de execução. Isso não significa necessariamente dez usuários simultâneos.

7.3. Criando a função principal do teste

```
export default function () {
  // Código do teste de performance
}
```

A função export default function() é a função principal executada pelo k6 em cada iteração do cenário padrão.

É dentro dela que serão realizadas as requisições HTTP, as verificações das respostas e as pausas entre as operações.

Um arquivo de script possui uma única exportação default. Para organizar testes diferentes, podem ser criados vários arquivos ou utilizadas outras funções exportadas conforme a necessidade.

8. Realizando uma requisição POST para login

A documentação apresenta exemplos de requisições GET e POST. Como o login da API utilizada na aula recebe uma requisição POST, é necessário consultar a documentação específica desse método.

A estrutura geral de uma requisição POST é:

```
const url = 'http://localhost:3000/login';

const payload = JSON.stringify({
  // Dados de login
});

const params = {
  headers: {
    'Content-Type': 'application/json',
  },
};

const response = http.post(url, payload, params);
```

Atenção: os dados do payload devem corresponder ao contrato real da API. O exemplo acima demonstra a estrutura, mas ainda não define usuário e senha, nem valida o resultado do login.

Elementos importantes:

- **url:** endereço do endpoint que será testado.
- **payload:** corpo enviado na requisição.
- **JSON.stringify():** transforma o objeto JavaScript em uma string JSON.
- **params:** permite configurar os cabeçalhos e outras opções da requisição.
- **Content-Type:** application/json: informa que o corpo está no formato JSON.
- **http.post():** envia a requisição HTTP POST.
- **response:** armazena a resposta recebida do servidor.

O endpoint http://localhost:3000/login pressupõe que a API esteja em execução localmente na porta 3000.

9. Utilizando o sleep()

A função sleep() permite aguardar um intervalo em segundos antes de continuar a execução.

Exemplo: **sleep(1);**

Nesse caso, o script aguarda aproximadamente um segundo.

O intervalo pode ser utilizado para simular pausas entre ações de um usuário. Em testes de performance, essas pausas devem refletir o comportamento que se deseja simular, pois influenciam a carga gerada.

10. Ponto e vírgula em JavaScript

O instrutor explica que o JavaScript permite escrever código sem ponto e vírgula em muitos casos.
A decisão de utilizá-lo deve seguir o padrão definido pela equipe ou pelo projeto. O mais importante é manter a consistência do código.

Conclusão

Nesta aula, foi iniciada a construção do projeto banco-api-performance, desde a criação do repositório no GitHub e da branch de desenvolvimento até a criação do primeiro arquivo JavaScript para o k6.

Os principais aprendizados foram:

- Versionar o projeto com Git e GitHub.
- Organizar os scripts em arquivos .js.
- Entender que o k6 executa os scripts por meio do comando k6, sem exigir um projeto Node.js convencional.
- Conhecer a estrutura básica de um script: importações, opções e função principal.
- Consultar a documentação oficial para aprender a utilizar requisições HTTP.
- Preparar uma requisição POST para o endpoint de login.

---

8. ADICIONANDO CHECKS NOS TESTES

1. Objetivo da aula

Aprender a utilizar a função check() do Grafana k6 para validar as respostas recebidas durante a execução dos testes de performance.

Essas verificações ajudam a confirmar que o teste está exercitando o cenário esperado, evitando conclusões incorretas sobre o desempenho da aplicação.

2. Testes funcionais x testes de performance

Os testes funcionais verificam se o sistema apresenta o comportamento esperado, de acordo com os requisitos e as regras de negócio.

Os testes de performance avaliam características não funcionais, como tempo de resposta, capacidade e utilização de recursos.

Aspecto	Testes funcionais	Testes de performance
Objetivo principal	Verificar o comportamento do sistema	Avaliar o desempenho sob determinadas condições
Exemplos de validação	Login válido, token retornado, regras de negócio	Tempo de resposta, taxa de requisições, erros e capacidade
Critério de sucesso	Comportamento esperado conforme os requisitos	Métricas de desempenho dentro dos limites definidos

Embora tenham objetivos diferentes, os testes de performance podem incluir verificações funcionais básicas para confirmar que as requisições estão produzindo resultados válidos.

3. Por que utilizar checks em testes de performance?

Imagine que o endpoint de login deveria retornar sucesso, mas está indisponível e retorna um erro imediatamente.

Se avaliarmos apenas o tempo de resposta, poderemos concluir que o sistema está respondendo muito rápido. Na realidade, o cenário esperado não foi executado com sucesso.

Por isso, os checks ajudam a verificar se:

- O status HTTP retornado é o esperado.
- O corpo da resposta contém os dados previstos.
- O token retornado possui o tipo esperado.
- O teste está exercitando o cenário correto.

Importante: um check não transforma o teste de performance em um teste funcional completo. Ele adiciona verificações pontuais para ajudar a interpretar os resultados corretamente.

4. Conhecendo a função check()

A função check() faz parte do módulo k6 e permite verificar condições booleanas durante a execução. Documentação oficial: grafana.com

.

Importação

Como o script já importa sleep do módulo k6, podemos importar as duas funções na mesma declaração:
import { sleep, check } from 'k6';

Estrutura básica

```
check(res, {
  'Status é 200': (r) => r.status === 200,
```

});

A função recebe dois argumentos:

1. res: a resposta que será avaliada.
2. Um objeto JavaScript com os checks que serão executados.

Nesse exemplo:

- 'Status é 200' é o nome da verificação.
- (r) => r.status === 200 é a função que avalia a condição.
- r representa a resposta recebida.
- r.status acessa o status HTTP.
- === 200 verifica se o status é exatamente 200.

O parâmetro r poderia ter outro nome, desde que fosse utilizado consistentemente dentro da função.

5. Validando o status HTTP

No exemplo da aula, o endpoint de login deve retornar o status HTTP 200 em caso de sucesso.

Após realizar a requisição, podemos verificar esse resultado:

```
const res = http.post(url, payload, params);

check(res, {
  'Status é 200': (r) => r.status === 200,
});
```

O check retorna true quando a condição é atendida e false quando não é.
Isso permite acompanhar quantas verificações passaram ou falharam durante o teste.

6. Validando o tipo do token

Além do status, a aula demonstra como verificar se o token retornado pelo login é uma string.

Exemplo:

```
check(res, {
  'Status é 200': (r) => r.status === 200,
  'Token é uma string': (r) => typeof r.json().token === 'string',
});
```

Entendendo a expressão:

- r.json() acessa o corpo da resposta interpretado como JSON.
- .token acessa a propriedade token.
- typeof retorna o tipo do valor.
- === 'string' verifica se o tipo é string.

Essa validação pressupõe que o corpo da resposta seja um JSON válido e que o token esteja na propriedade token.

7. Diferença entre check() e asserções tradicionais

Em ferramentas de testes funcionais, as asserções normalmente verificam condições esperadas e podem interromper a execução do teste quando falham, dependendo da ferramenta e da configuração.

No k6, os checks registram o resultado da verificação, mas não interrompem automaticamente a execução quando falham.

Por exemplo, se o status for 401 em vez de 200, o check falhará, mas o script continuará executando as próximas instruções, salvo se houver outra lógica para interrompê-lo.

Isso é importante para coletar métricas e observar o comportamento do sistema durante todo o teste.

8. Cuidados ao interpretar os resultados

Ao analisar os resultados do k6, não devemos considerar apenas o tempo de resposta.

Também é importante observar:

- A quantidade de checks aprovados e reprovados.
- A taxa de erros HTTP.
- Os tempos de resposta, incluindo percentis.
- Se o cenário executado corresponde ao cenário planejado.

Um tempo de resposta baixo acompanhado de falhas pode indicar que o sistema não está processando corretamente as requisições esperadas.

Conclusão

Nesta aula, aprendemos a utilizar a função check() para adicionar verificações ao script de performance com k6.

Os principais aprendizados foram:

- Diferenciar o objetivo dos testes funcionais e dos testes de performance.
- Entender por que verificações pontuais ajudam a evitar interpretações incorretas dos resultados.
- Importar check do módulo k6.
- Validar o status HTTP retornado pelo endpoint.
- Verificar se o token retornado é uma string.
- Entender que os checks registram falhas sem interromper automaticamente a execução.

O próximo passo é executar o script com os checks implementados e observar os resultados apresentados pelo k6, verificando tanto o desempenho quanto a validade das respostas recebidas.

---

9. IMPLEMENTANDO THRESHOLDS

1. Objetivo da aula

Aprender a implementar thresholds no Grafana k6 para definir critérios objetivos de aprovação e reprovação dos testes de performance.

Enquanto os checks verificam condições específicas das respostas, os thresholds estabelecem limites para as métricas de desempenho e podem fazer com que a execução do teste termine com falha quando esses limites não forem atendidos.

2. O que são thresholds?

Thresholds são critérios mensuráveis que determinam se os resultados de um teste de performance estão dentro dos limites estabelecidos.

Esses critérios devem ser definidos com base nos requisitos não funcionais da aplicação ou alinhados com a equipe responsável pelo sistema, como o Product Owner, o Tech Lead ou o time de QA.

Exemplo apresentado na aula:

- Uma API deve responder em até 3.000 milissegundos (3 segundos).
- O percentil 95 (P95) deve permanecer abaixo desse limite.
- Se o P95 ultrapassar o limite definido, o threshold falha.

O threshold não é apenas um valor isolado: ele associa uma métrica a uma condição que precisa ser satisfeita.

3. Exemplos de critérios de performance

A aula apresenta exemplos de thresholds que podem ser definidos para uma API:

Critério	**Significado**
http_req_failed menor que 0.01	Menos de 1% das requisições falharam
http_req_duration P95 abaixo de 200 ms	95% das requisições tiveram duração de até aproximadamente 200 ms, conforme o limite configurado
http_req_duration P99 abaixo de 400 ms	99% das requisições tiveram duração de até aproximadamente 400 ms
Duração máxima abaixo de um limite definido	Nenhuma requisição ultrapassou o limite máximo estabelecido

Os valores são exemplos didáticos. Na prática, os limites devem refletir os requisitos e as necessidades do sistema.

4. Como configurar thresholds no k6

Os thresholds são definidos na propriedade thresholds, dentro do objeto options, utilizando as métricas do k6.

Exemplo:

```
export const options = {
  iterations: 10,

  thresholds: {
    http_req_duration: ['p(90)<10'],
  },
};
```

Nesse exemplo:

- thresholds: define os critérios de aprovação do teste.
- http_req_duration: identifica a métrica de duração das requisições HTTP.
- p(90)<10: exige que o percentil 90 seja inferior a 10 milissegundos.

A sintaxe p(90) representa o percentil 90. O limite de 10 ms foi utilizado na aula apenas para demonstrar o funcionamento dos thresholds, e não como recomendação geral para APIs.

Documentação oficial: grafana.com

.

5. Entendendo as métricas utilizadas

A aula trabalha principalmente com duas métricas:

Métrica	**Significado**
http_req_duration	Duração das requisições HTTP, considerando o envio, a espera pela resposta e o recebimento dos dados, sem incluir o tempo de bloqueio inicial e a conexão TCP/TLS
http_req_failed	Taxa de requisições HTTP consideradas malsucedidas conforme a classificação do k6

Para http_req_duration, podemos utilizar diferentes medidas estatísticas:

- avg: média.
- min: menor duração.
- max: maior duração.
- med: mediana.
- p(90): percentil 90.
- p(95): percentil 95.

A escolha da medida depende do requisito de performance. A aula destaca o uso de percentis como prática frequente.

6. Testando um threshold que falha

A proposta do instrutor é definir intencionalmente um limite muito restritivo para observar a reprovação.

Por exemplo:

```
thresholds: {
  http_req_duration: ['p(90)<1'],
},
```

Esse threshold exige que o P90 seja inferior a 1 milissegundo. Se o resultado for maior ou igual a 1 ms, o critério não será atendido.

Como a duração das requisições pode variar de uma execução para outra, um teste pode passar em uma execução e falhar em outra quando o resultado estiver próximo do limite.

A aula demonstra que, quando um threshold é violado, o k6 apresenta uma mensagem indicando a métrica e o critério que falhou.

Importante: um limite artificialmente baixos serve para fins didáticos. Em um projeto real, os thresholds devem ser definidos a partir de requisitos mensuráveis.

7. Configurando mais de um threshold para a mesma métrica

É possível definir vários critérios para http_req_duration.

Exemplo:

```
export const options = {
  iterations: 10,

  thresholds: {
    http_req_duration: [
      'p(90)<10',
      'max<100',
    ],
  },
};
```

Nesse caso, o teste precisa satisfazer os dois critérios:

1. O P90 deve ser inferior a 10 ms.
2. A maior duração registrada deve ser inferior a 100 ms.

Se qualquer um dos critérios falhar, o threshold dessa métrica não será atendido.

8. Configurando thresholds para métricas diferentes

Também é possível avaliar várias métricas no mesmo teste.

Exemplo:

```
export const options = {
  iterations: 10,

  thresholds: {
    http_req_duration: [
      'p(90)<10',
      'max<100',
    ],

    http_req_failed: [
      'rate<0.01',
    ],
  },
};
```

Nesse exemplo, o teste exige:

- P90 abaixo de 10 ms.
- Duração máxima abaixo de 100 ms.
- Taxa de falhas inferior a 1%.

A condição rate<0.01 significa que a taxa de falhas deve ser inferior a 1%. Portanto, ela não exige necessariamente zero falhas.

Se o requisito for não permitir nenhuma falha, a condição poderá ser definida como:
http_req_failed: ['rate==0']

9. Diferença entre checks e thresholds

Esse é um dos conceitos mais importantes da aula.

Característica	**Checks**	**Thresholds**
Finalidade	Verificar condições específicas de uma resposta	Definir limites para métricas do teste
Exemplo	Status HTTP igual a 200	P95 abaixo de 3.000 ms
Quando falha	Registra a verificação como reprovada	Pode fazer a execução do teste terminar com falha
Interpretação	Ajuda a confirmar se o cenário está funcionando como esperado	Determina se os critérios de performance foram atendidos

A aula demonstra essa diferença ao interromper intencionalmente a API durante a execução:

- Os checks começam a falhar porque as respostas deixam de atender às condições esperadas.
- A métrica http_req_failed aumenta.
- O threshold configurado para a taxa de falhas deixa de ser atendido.
- A execução termina com falha devido à violação do threshold.

10. Boas práticas

- Definir thresholds com base nos requisitos não funcionais.
- Priorizar métricas relevantes para o comportamento esperado da aplicação.
- Utilizar percentis para avaliar a distribuição dos tempos de resposta.
- Configurar limites coerentes para duração e taxa de falhas.
- Investigar os motivos de reprovação, em vez de apenas observar o resultado final.
- Evitar utilizar valores didáticos como critérios definitivos em ambientes reais.

Conclusão

Nesta aula, aprendemos a implementar thresholds no k6 para transformar requisitos de performance em critérios objetivos de aprovação e reprovação.

Os principais aprendizados foram:

- Entender o conceito de threshold e sua relação com os requisitos não funcionais.
- Configurar a propriedade thresholds dentro de options.
- Utilizar métricas como http_req_duration e http_req_failed.
- Definir critérios usando média, máximo e percentis.
- Configurar vários thresholds para uma mesma métrica ou para métricas diferentes.
- Diferenciar checks, que registram o resultado de verificações, de thresholds, que podem reprovar a execução do teste.

Conceito central: os checks ajudam a verificar se as respostas estão corretas; os thresholds definem se o desempenho observado atende aos limites estabelecidos.

---

10. GERANDO RELATÓRIOS DE EXECUÇÃO

1. Para que serve o Web Dashboard?

O Web Dashboard apresenta gráficos e informações que ajudam a analisar os resultados dos testes de performance, como:

- Quantidade de requisições executadas.
- Taxa de requisições por segundo.
- Tempo de resposta.
- Quantidade de usuários virtuais (VUs).
- Falhas nas requisições.
- Resultados dos thresholds configurados.

Essas informações permitem avaliar o comportamento da aplicação durante o teste de carga.

2. Existem duas formas de gerar o relatório

Forma 1 — Dashboard em tempo real
Antes do comando de execução do k6, adicione a variável de ambiente:

K6_WEB_DASHBOARD=true k6 run tests.js

Substitua tests.js pelo nome do arquivo de teste utilizado no seu projeto.

Como funciona:

1. Inicie a API que será testada.
2. Execute o comando com a variável K6_WEB_DASHBOARD=true.
3. O k6 exibirá um endereço local no terminal.
4. Abra esse endereço no navegador para visualizar os gráficos.

O dashboard funciona durante a execução do teste. Portanto, você precisa acessá-lo enquanto o teste estiver rodando para acompanhar os resultados em tempo real.

Forma 2 — Exportar o relatório em HTML

Para gerar um arquivo HTML que possa ser consultado posteriormente, utilize:
K6_WEB_DASHBOARD=true K6_WEB_DASHBOARD_EXPORT=relatorio.html k6 run tests.js

O arquivo relatorio.html será gerado com o relatório da execução, permitindo abri-lo no navegador depois que o teste terminar.

3. Diferença entre as duas formas

Característica	**Dashboard em tempo real**	**Relatório HTML**
Acompanhamento durante o teste	Sim	Não é necessário
Visualização no navegador	Sim	Sim
Consulta após o teste	Não, pelo endereço temporário do dashboard	Sim, pelo arquivo exportado
Arquivo salvo	Não necessariamente	Sim, em HTML

5. **Cuidados durante a execução**

- **API ligada:** a aplicação que será testada precisa estar em execução.
- **Thresholds:** se você configurar limites impossíveis de alcançar, o teste poderá apresentar falhas nos thresholds. Isso não significa necessariamente que a API parou de funcionar.
- **Duração do teste:** uma execução muito rápida pode dificultar a visualização do dashboard em tempo real. Aumentar a duração pode ajudar.
- **Relatório HTML:** confira o terminal e o local de execução para localizar o arquivo exportado.

5. O que você precisa memorizar para QA

O ponto principal desta aula é entender a diferença entre acompanhar a execução em tempo real e exportar resultados para análise posterior.

- K6_WEB_DASHBOARD=true: habilita o dashboard.
- K6_WEB_DASHBOARD_EXPORT=relatorio.html: define o arquivo HTML de saída.
- k6 run: executa o teste de performance.

Exemplo de uso combinado:
K6_WEB_DASHBOARD=true \
K6_WEB_DASHBOARD_EXPORT=relatorio.html \
k6 run tests.js

Esse comando habilita o dashboard e exporta o relatório HTML ao final da execução.
Lembre-se: os relatórios auxiliam na análise não funcional do sistema, especialmente em relação ao desempenho, ao tempo de resposta e à capacidade de suportar carga. Eles complementam os resultados dos testes, mas não substituem a análise dos critérios de aceitação e dos thresholds configurados.

---

11. VERSIONANDO NOSSO SCRIPT DE TESTE DE PERFORMANCE

1. Por que versionar o script?

O versionamento permite:
- Manter o histórico das alterações realizadas no script.
- Recuperar versões anteriores do código.
- Acompanhar a evolução dos testes.
- Compartilhar o código com outras pessoas.
- Trabalhar com segurança nas próximas modificações.

Para quem trabalha com QA, isso é importante tanto em projetos de automação funcional quanto em testes não funcionais, como os testes de performance com k6.

2. Passo a passo executado na aula

Etapa 1 — Verificar as alterações

git status
Esse comando mostra os arquivos modificados, novos arquivos e outras alterações ainda não registradas no Git.
Na aula, foram identificados o relatório HTML e a pasta tests.

Etapa 2 — Ignorar o relatório HTML

Foi criado o arquivo .gitignore, com o seguinte conteúdo:
html-report.html
O objetivo é impedir que o relatório temporário seja incluído no versionamento.
Atenção: o nome da regra precisa corresponder ao nome real do arquivo gerado. Se ele estiver em uma subpasta, a regra também poderá precisar ser ajustada.

O .gitignore evita que arquivos desnecessários apareçam como novos arquivos não rastreados. Ele não deixa de rastrear automaticamente arquivos que já foram adicionados ao Git anteriormente.

Etapa 3 — Adicionar os arquivos ao stage

git add .
Esse comando prepara as alterações do diretório atual para serem incluídas no próximo commit.

Etapa 4 — Conferir o stage

git status
Verifique se os arquivos que você deseja versionar aparecem na seção de alterações preparadas para commit.

Etapa 5 — Criar o commit
git commit -m "Adiciona arquivo inicial com teste de login"
O commit registra uma versão do código no histórico local do Git.

Etapa 6 — Identificar a branch atual
git branch
O comando mostra as branches locais. O asterisco (*) identifica a branch atual.

Etapa 7 — Enviar para o repositório remoto
git push origin nome-da-branch
Substitua nome-da-branch pelo nome real da branch utilizada no projeto.
O comando envia os commits locais para o repositório remoto, desde que a branch e as permissões estejam configuradas corretamente.

3. Fluxo de versionamento para memorizar

1. git status
Verificar as alterações
2. git add .
Preparar os arquivos (stage)
3. git commit -m "mensagem"
Registrar uma versão local
4. git push origin nome-da-branch
Enviar os commits ao remoto

4. Conceitos importantes para QA

Conceito	**Significado**
.gitignore	Define arquivos e pastas que o Git deve ignorar quando ainda não estão sendo rastreados.

git add	Prepara alterações para o commit.
Stage	Área de preparação das alterações.
git commit	Registra uma versão no histórico local.
Branch	Linha de desenvolvimento do projeto.
git push	Envia commits para um repositório remoto.
Repositório remoto	Cópia hospedada do projeto, por exemplo, no GitHub.

---

12. ADICIONANDO USUÁRIOS VIRTUAIS

1. O que são usuários virtuais (VUs)?
Os Virtual Users (VUs) são usuários virtuais que executam o script de teste, permitindo simular acessos concorrentes à aplicação.

A diferença em relação ao que você estudou anteriormente é:
Iterations — execuções do script
O teste é repetido uma quantidade definida de vezes. Por exemplo, 50 iterações.
A quantidade de execuções é definida previamente.

VUs + Duration — usuários virtuais durante um período
Por exemplo, 10 usuários virtuais executando o teste durante 30 segundos.
A quantidade total de iterações e requisições depende do comportamento do script e do ambiente.

2. Configurando VUs e duração no script
No arquivo de teste de login, a configuração das opções fica no objeto options.

Exemplo:
```
export const options = {
  vus: 10,
  duration: '30s',

  thresholds: {
    http_req_duration: [
      'p(90)<3000',
      'max<5000'
    ],
    http_req_failed: [
      'rate<0.01'
    ]
  }
};
```

O que cada configuração significa:

Configuração	Significado
vus: 10	Define 10 usuários virtuais.
duration: '30s'	Define 30 segundos de duração para o cenário de execução contínua.
p(90)<3000	Exige que o percentil 90 do tempo de resposta seja inferior a 3.000 ms.
max<5000	Exige que o máximo observado seja inferior a 5.000 ms.
rate<0.01	Exige uma taxa de falhas inferior a 1%.

Importante: esses thresholds são exemplos da aula, não valores universais. Os limites corretos devem vir dos requisitos de desempenho da aplicação.

3. Como funciona a execução?



Os VUs não precisam iniciar todos exatamente no mesmo instante. O k6 agenda sua execução, e o comportamento real depende da configuração do cenário, do script e dos recursos disponíveis.

4. Por que foram registradas 300 requisições?

Na execução demonstrada, o script continha uma espera de aproximadamente 1 segundo por iteração, e foram usados 10 VUs durante 30 segundos.

A estimativa didática é:

$$10 \times 30 = 300$$

Isso explica o total observado no exemplo, mas não é uma regra fixa. Se houver outras esperas, requisições adicionais, processamento demorado ou falhas, o resultado pode ser diferente.

Além disso, o tempo de sleep() não é necessariamente o tempo de resposta HTTP: são medidas distintas.

5. Entendendo as métricas do relatório

Métrica	**O que avaliar**
http_req_duration	Tempo gasto na requisição HTTP, incluindo envio, espera e recebimento, conforme a métrica do k6.
avg	Tempo médio de resposta.
med	Mediana: metade das observações está abaixo ou igual a esse valor.
p(90)	Percentil 90: 90% das observações estão abaixo ou igual a esse valor.
p(95)	Percentil 95: 95% das observações estão abaixo ou igual a esse valor.
p(99)	Percentil 99: 99% das observações estão abaixo ou igual a esse valor.
max	Maior tempo de resposta observado.
http_req_failed	Proporção de requisições HTTP consideradas malsucedidas pelo k6.
http_reqs	Total de requisições HTTP realizadas.
iterations	Total de iterações concluídas.

Exemplo de análise de QA
Imagine que o requisito da aplicação seja p(95) < 15 ms.

- Resultado de 11 ms: atende ao limite de desempenho, considerando essa métrica.
- Resultado de 17 ms: não atende ao limite.

A avaliação completa também precisa verificar falhas, volume de carga, duração do teste e demais critérios definidos.

6. Versionando as alterações no Git

Depois de modificar o script, a aula registra a nova versão no Git.

```
git status
git add .
git status
git commit -m "Adiciona VUs e duration à execução dos testes"
git push origin nome-da-branch
```

O fluxo é:

1. git status: conferir as alterações.

2. git add .: preparar os arquivos.
3. git status: verificar o stage.
4. git commit: registrar a versão local.
5. git push: enviar a versão para o repositório remoto.

No GitHub, a opção History permite consultar os commits anteriores e visualizar as versões históricas do script.

7. O que você precisa guardar desta aula

- **VUs:** usuários virtuais que executam o script.
- **Duration:** duração definida para a execução do cenário.
- **Iterations:** quantidade de iterações realizadas.
- **Thresholds:** critérios objetivos usados para determinar se os resultados atendem aos requisitos.
- **Relatório HTML:** permite analisar os resultados após a execução.
- **Git:** registra a evolução do código de teste.

Uma observação importante para sua formação em QA: usar 10 VUs durante 30 segundos é um bom exercício inicial, mas ainda não representa necessariamente um teste de carga realista. Para isso, precisamos definir uma carga compatível com o uso esperado da aplicação e analisar se o sistema mantém o desempenho exigido.

---

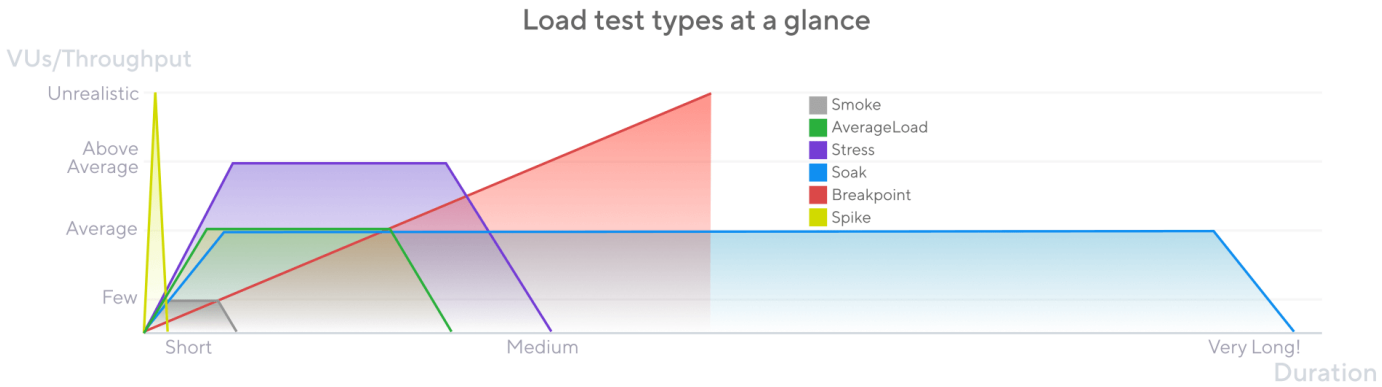
13. TESTE DE PERFORMANCE COM K6

1. O que são stages?

Os stages são estágios de execução definidos no script do k6. Cada estágio possui duas propriedades principais:

- duration: tempo de duração do estágio.
- target: quantidade de usuários virtuais que o k6 deverá atingir ao final daquele estágio.

Por exemplo, você pode começar com 10 usuários, aumentar para 30 e depois reduzir gradualmente até zero.



2. Como configurar stages no código

No arquivo tests/login.test.js, a configuração pode ficar assim:

JavaScript

```javascript
export const options = {
    stages: [
        { duration: '10s', target: 10 },
        { duration: '20s', target: 10 },
        { duration: '10s', target: 30 },
        { duration: '20s', target: 30 },
        { duration: '20s', target: 0 },
    ],

    thresholds: {
        http_req_duration: [
            'p(90)<3000',
            'max<5000',
        ],
        http_req_failed: ['rate<0.1'],
    },
};
```

Essa configuração mantém os thresholds do exemplo da aula. Os limites devem ser ajustados conforme os requisitos reais do sistema.

Importante: quando você utiliza essa configuração, não precisa definir vus e duration no mesmo nível para controlar a carga desse cenário. Os stages passam a definir a evolução dos usuários virtuais ao longo do teste.

3. Entenda a sequência dos estágios



A duração total configurada é de **80 segundos**, sem considerar o tempo adicional de encerramento gracioso das iterações.

4. Como visualizar os stages no dashboard

1. Execute seu teste com o dashboard habilitado.
2. Abra http://127.0.0.1:5665 enquanto o k6 estiver executando.
3. Localize o gráfico de usuários virtuais, identificado como VUs.
4. Observe o gráfico aumentar, permanecer estável e diminuir conforme os estágios configurados.

O gráfico pode apresentar pequenas variações durante as transições. O importante é analisar a carga planejada, a execução efetiva e as métricas resultantes.

5. Relação entre stages e tipos de testes de performance

Os stages são uma ferramenta de configuração de carga. Eles ajudam a implementar diferentes estratégias de teste, dependendo de como você os organiza.

Tipo de teste	**Objetivo**	**Exemplo de estratégia**
Smoke test	Verificar se o sistema funciona com carga mínima.	Poucos VUs por pouco tempo.
Load test	Avaliar o desempenho sob carga esperada.	Aumentar até a carga normal e mantê-la.
Stress test	Investigar o comportamento além da carga esperada.	Aumentar progressivamente os VUs acima da carga normal.
Spike test	Avaliar a reação a um aumento abrupto de carga.	Subir rapidamente de poucos para muitos VUs.
Soak test	Identificar problemas sob carga prolongada.	Manter uma carga representativa por um período longo.

Essas estratégias não dependem exclusivamente de stages: também envolvem a carga escolhida, a duração, os objetivos e os critérios de aprovação.

6. O que guardar para sua atuação em QA

- **stages** define como a carga evolui durante o teste.
- **duration** determina o tempo de cada estágio.
- **target** define a quantidade de VUs desejada ao final do estágio.
- **thresholds** define critérios mensuráveis para avaliar os resultados.
- O **dashboard** ajuda a comparar a carga planejada com a execução observada.

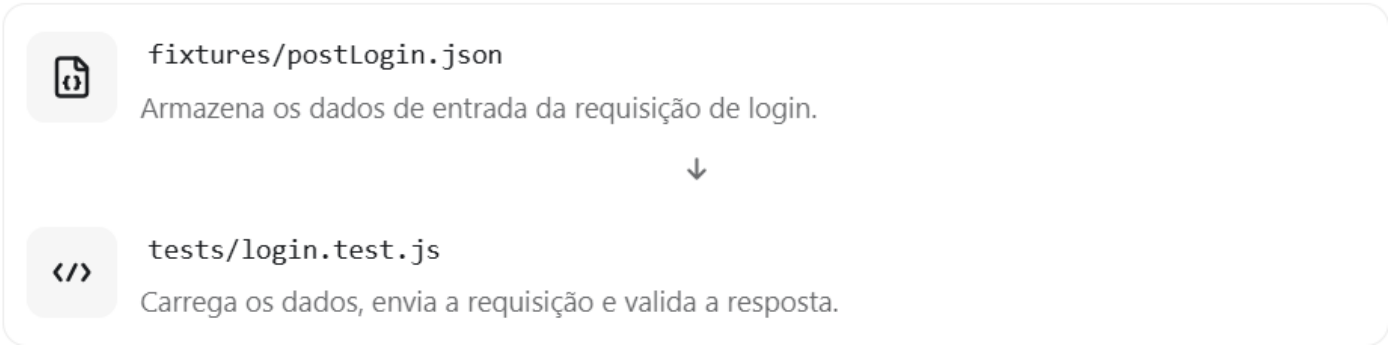
Próximo passo: ao estudar os tipos de testes de performance, procure relacionar cada estratégia ao cenário de negócio que ela pretende representar. Isso é o que permite justificar, como profissional de QA, por que determinado perfil de carga foi escolhido e quais resultados devem ser avaliados.

---

15. SEGREGANDO ARQUIVOS DE DADOS DO TESTE

1. Qual é o objetivo?

Separar os dados da requisição do código responsável por executar o teste.



Essa separação facilita a manutenção, a reutilização dos dados e a criação de diferentes cenários de teste.

2. Estrutura dos arquivos

O projeto fica organizado assim:

Texto simples

```
3-teste-de-performance-k6/
├── fixtures/
│   └── postLogin.json
├── tests/
│   └── login.test.js
└── html-report.html
```

O arquivo html-report.html só estará presente se o relatório tiver sido gerado com sucesso.

3. Criando o arquivo JSON

Arquivo fixtures/postLogin.json:

JSON

```json
{
  "username": "junior.lima",
  "senha": "string"
}
```

Atenção às diferenças entre JSON e objeto JavaScript:

- As propriedades do JSON precisam estar entre aspas duplas.

- Os valores do tipo texto também usam aspas duplas.
- O JSON não aceita comentários nem vírgulas sobrando no final das propriedades.

4. Importando os dados no teste

No arquivo tests/login.test.js, usamos:

```javascript
const postLogin = JSON.parse(open('../fixtures/postLogin.json'));
```

- open() lê o arquivo externo.
- JSON.parse() transforma o conteúdo JSON em um objeto JavaScript.
- postLogin permite acessar e modificar as propriedades do objeto durante o teste.

O caminho ../fixtures/postLogin.json significa sair da pasta tests e entrar na pasta fixtures.

5. Modificando os dados sem alterar o arquivo JSON

Por exemplo:

```javascript
postLogin.username = 'junior.lima';

console.log(postLogin);
```

A alteração acontece no objeto carregado em memória. O arquivo JSON original permanece inalterado. Isso permite executar testes com diferentes dados sem precisar editar o arquivo a cada execução.

6. Executando apenas uma iteração

Para testar rapidamente se o arquivo JSON foi carregado corretamente, você pode configurar:

```javascript
export const options = {
    iterations: 1,
};
```

Deixe os stages comentados durante essa verificação. Depois, você pode reativar os estágios para executar o teste de performance.

Importante: iterations: 1 significa uma iteração total no cenário padrão. Já vus define a quantidade de usuários virtuais.

7. Aplicando ao seu projeto atual

Pelos resultados que você compartilhou, seu script já consegue ler o JSON e enviar uma requisição para a API. A resposta atual foi:

```JSON
{
  "error": "Usuário ou senha inválidos."
}
```

Isso indica que o servidor recebeu a requisição, mas recusou a autenticação. O próximo passo é verificar as credenciais válidas definidas pela aplicação e confirmar o formato esperado pelo endpoint.

Não altere o threshold para esconder essa falha: primeiro precisamos garantir que o cenário de login bem-sucedido funcione.

8. Versionando as alterações com Git

Depois de validar o código, a aula orienta registrar as alterações no Git:
git status
git add .
git status
git commit -m "Adiciona fixtures para dados de entrada do teste"
git push origin teste-performance

Use o nome real da sua branch. Pelo terminal que você compartilhou, ela é teste-performance.

O que você aprendeu nesta aula

- Separar os dados de entrada do código de teste.
- Criar arquivos JSON para corpos de requisição.
- Carregar arquivos externos com open() e JSON.parse().
- Modificar os dados em memória para testar diferentes cenários.
- Executar uma iteração para validar a configuração.
- Versionar as alterações com Git.

Essa técnica é útil tanto em testes funcionais de API quanto em testes de performance com k6. A diferença é o objetivo: aqui você está validando a organização e o comportamento do teste; na etapa de performance, também avaliará como o sistema se comporta sob carga.

---

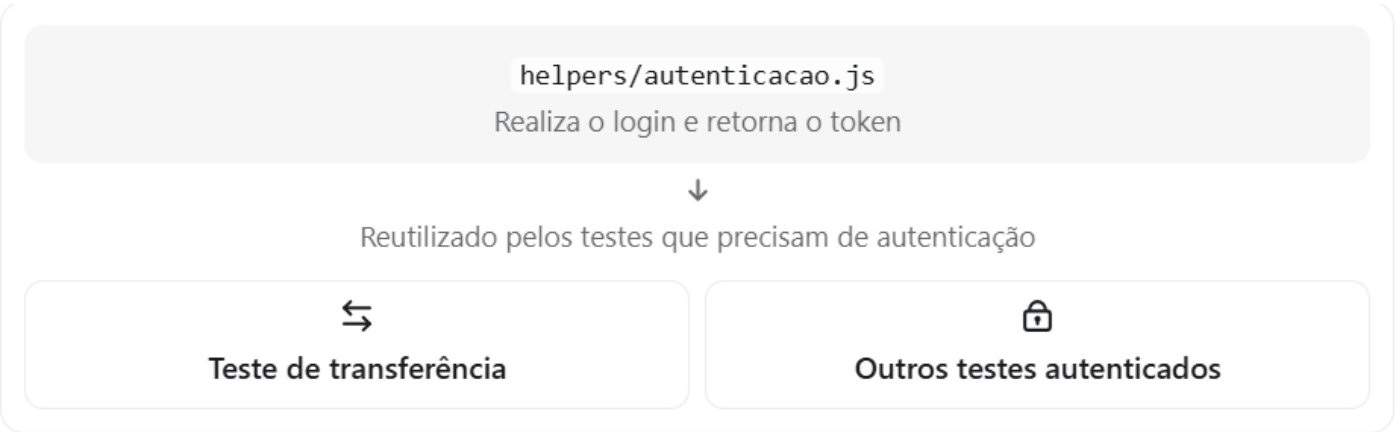
16. REAPROVEITANDO CÓDIGO E USANDO O TOKEN

1. Qual problema estamos resolvendo?

Para executar uma transferência, a API exige um token de autenticação, obtido por meio do login.

Uma solução ruim seria copiar o código de login para cada teste que exige autenticação. Isso gera duplicação e dificulta a manutenção.

A solução é criar um helper que centraliza a obtenção do token.



2. Estrutura do projeto

Ao final da aula, a estrutura deverá ficar semelhante a esta:

Texto simples

```
3-teste-de-performance-k6/
├── fixtures/
│   └── postLogin.json
├── helpers/
│   └── autenticacao.js
└── tests/
    ├── login.test.js
    └── transferencias.test.js
```

Observe que helpers fica na raiz do projeto, no mesmo nível de fixtures e tests.

3. Criando o helper de autenticação

No arquivo helpers/autenticacao.js, a lógica é:

1. Importar o módulo HTTP do k6.
2. Carregar os dados do login pelo arquivo JSON.
3. Enviar a requisição para o endpoint de login.
4. Extrair o token da resposta.
5. Retornar somente o token.

Exemplo:

```
import http from 'k6/http';

const postLogin = JSON.parse(open('../fixtures/postLogin.json'));

export function obterToken() {
  const url = 'http://localhost:3000/login';

  const params = {
    headers: {
      'Content-Type': 'application/json',
    },
  };

  const resposta = http.post(
    url,
    JSON.stringify(postLogin),
    params
  );

  return resposta.json('token');
}
```

Esse exemplo pressupõe que o endpoint de login retorna HTTP 200 e um campo token na raiz do JSON, conforme explicado na aula. Se o helper do seu projeto já existe, compare-o com esse exemplo antes de substituir qualquer código.

Entenda a linha principal

```JavaScript
return resposta.json('token');
```

- resposta é o resultado da requisição HTTP.
- .json('token') extrai o campo token do corpo JSON.
- return devolve esse valor para quem chamou a função.

Assim, o teste recebe o token sem precisar repetir o código de autenticação.

4. Utilizando o helper no teste de transferência

No arquivo tests/transferencias.test.js, importamos a função:

```JavaScript
import { obterToken } from '../helpers/autenticacao.js';
```

Em seguida, chamamos a função e usamos o valor retornado no cabeçalho:

```JavaScript
const token = obterToken();

const params = {
    headers: {
        'Content-Type': 'application/json',
        authorization: `Bearer ${token}`,
    },
};
```

O formato Bearer é utilizado para enviar o token de autenticação. O token não deve ser fixado no código porque pode expirar.

Atenção: confirme o status esperado no contrato da API. Na transcrição, o teste de transferência é descrito como validando HTTP 200, enquanto no seu teste anterior você validava 201. O status correto depende da implementação real do endpoint.

5. Por que usar helpers?

Sem helper	Com helper
Código de login duplicado	Login centralizado
Alterações em vários testes	Alteração em um único lugar
Mais difícil de manter	Mais fácil de reutilizar
Mais repetição	Melhor organização

Outro ponto importante para testes de performance: cada chamada a obterToken() executa uma nova requisição de login. Isso deve ser considerado ao analisar as métricas, pois o tempo e a carga do login também podem entrar nos resultados globais do teste.

6. Executando o teste

No Git Bash, a partir da raiz do projeto: k6 run tests/transferencias.test.js

Para usar o dashboard e tentar exportar o relatório:
K6_WEB_DASHBOARD=true K6_WEB_DASHBOARD_EXPORT=html-report.html k6 run tests/transferencias.test.js

Se o script tiver apenas uma iteração e terminar em cerca de um segundo, a exportação HTML poderá ser ignorada por falta de dados suficientes. Isso não significa, por si só, que o teste falhou.

7. Versionando com Git

Depois de verificar o resultado:
git status
git add .
git status
git commit -m "Adiciona helper de autenticacao e teste de transferencia"
git push origin teste-performance
Confira o nome da branch com git branch --show-current se tiver dúvidas.

O que você deve guardar desta aula

- Fixture: arquivo externo com dados de entrada, como postLogin.json.
- Helper: função reutilizável que encapsula uma ação, como obter um token.
- Importação: permite usar a função auxiliar em outros arquivos.
- Retorno: return resposta.json('token') devolve apenas o token.
- Reutilização: evita duplicar o código de autenticação em cada teste.

A ideia central é construir testes menores, organizados e fáceis de manter. Essa prática também prepara seu projeto para crescer com novos cenários de API.

---

17. Variáveis de ambiente e arquivos de configuração

1. O que você vai implementar



2. Estrutura de pastas

A estrutura ficará assim:

Texto simples

```
3-teste-de-performance-k6/
├── config/
│   └── config.local.json
├── fixtures/
│   └── postLogin.json
├── helpers/
│   └── autenticacao.js
├── tests/
│   ├── login.test.js
│   └── transferencias.test.js
└── Utils/
    └── variaveis.js
```

Atenção: mantenha a grafia e as maiúsculas dos nomes de pastas iguais às utilizadas nos imports do seu projeto.

3. Criando a configuração local

Arquivo config/config.local.json:

```
{
  "baseURL": "http://localhost:3000"
}
```

Esse arquivo contém a URL padrão da API. Você poderá incluir outras configurações futuramente.

4. Criando o utilitário de variáveis

Arquivo Utils/variaveis.js:

```
const configLocal = JSON.parse(
   open('../config/config.local.json')
);

export function pegarBaseUrl() {
   return __ENV.BASE_URL || configLocal.baseURL;
}
```

A lógica é:

1. __ENV.BASE_URL busca a variável recebida pelo terminal.
2. || utiliza o valor à direita se o valor à esquerda for vazio ou não estiver definido.
3. configLocal.baseURL fornece a URL padrão do arquivo JSON.

Importante: o caminho passado para open() deve ser validado conforme o script principal que o k6 executa. Como o utilitário é importado por vários testes, vale executar o teste para confirmar que a fixture de configuração está sendo encontrada.

5. Executando com variável de ambiente

No Git Bash, dentro da pasta do projeto:
k6 run -e BASE_URL=http://localhost:3000 tests/transferencias.test.js

Para outro ambiente, você altera apenas o valor:
k6 run -e BASE_URL=https://servidor-de-teste.example tests/transferencias.test.js

O segundo endereço é ilustrativo: substitua pelo endereço real do ambiente de testes.

6. Usando a função nos testes

Nos arquivos tests/login.test.js e tests/transferencias.test.js, importe a função:

```JavaScript
import { pegarBaseUrl } from '../Utils/variaveis.js';
```

E monte a URL do endpoint:

```JavaScript
const url = pegarBaseUrl() + '/login';
```

No teste de transferência, utilize o caminho real do endpoint correspondente, por exemplo:
const url = pegarBaseUrl() + '/transferencias';

No arquivo helpers/autenticacao.js, faça a mesma substituição para que o login utilizado na obtenção do token também respeite o ambiente selecionado.

7. Qual é a prioridade da configuração?

Situação	**URL utilizada**
Executa com -e BASE_URL=...	Valor recebido pelo terminal
Executa sem -e BASE_URL=...	Valor de config.local.json
JSON ausente ou inválido	O teste pode falhar ao carregar a configuração

---

18. Criando o arquivos readme

Escolha uma inteligência artificial, passe o caminho do diretório no github e peça para criar o readme.

- Organização das seções, seguindo a estrutura de introdução, tecnologias, arquivos, instalação, execução e funcionalidades encontrada na documentação dos outros projetos.
- Explicação da responsabilidade de cada pasta e arquivo.
- Passo a passo para clonar o repositório, selecionar a branch e executar os testes.
- Descrição dos cenários de login e transferência.
- Explicação dos checks e thresholds de desempenho.
- Comandos para acompanhar o dashboard do k6 e gerar relatórios HTML separados para cada cenário.
- Correções de texto e orientações sobre erros, credenciais e interpretação dos resultados.
- Recomendações de segurança para credenciais e execução dos testes.

---

19. SÍNTESE DO APRENDIZADO

Os testes de performance avaliam o comportamento do sistema em diferentes condições de uso, considerando aspectos como tempo de resposta, capacidade e utilização de recursos.

O k6 permite automatizar esses testes por meio de scripts JavaScript. Para analisar os resultados, utilizamos métricas, checks e thresholds, além de conceitos como usuários virtuais (VUs), iterações e estágios de carga.

A organização dos scripts, a utilização de dados externos e helpers, a configuração por variáveis de ambiente e a geração de relatórios facilitam a manutenção e a análise dos testes.

O papel do QA é identificar gargalos, comparar os resultados com os requisitos não funcionais e fornecer evidências para melhorar o desempenho e a confiabilidade do sistema.
